

The QEC Challenge

A public, automatically verified leaderboard for quantum LDPC codes

Farrokh Labib, Vincent Russo, Ben Castanon, Bradley A. Chase,
Mathys Rennela, and William J. Zeng

Unitary Foundation

github.com/unitaryfoundation/qldpc-challenge

Extended abstract, lightning talk, Quantum Software 2.6 Workshop, IEEE Quantum Week
(IEEE QCE 2026), Metro Toronto Convention Centre, 17 September 2026

Abstract

Quantum low-density parity-check (qLDPC) codes are the leading route to low-overhead fault tolerance, but the field has no shared, machine-checked record of which codes are best under which constraints. Parameters are scattered across papers and tables, and distance claims in particular are hard to reproduce and easy to overstate, since computing distance is NP-hard and is often estimated with heuristic decoders that can be wrong. The QEC Challenge is a public leaderboard that treats this as a software-infrastructure problem. A participant submits one JSON file describing a CSS code; continuous integration recomputes the code’s properties from its parity-check matrices, checks a self-certifying distance witness, and merges the entry only if it holds up. Ranking is a per-track Pareto frontier rather than a single gameable number, track membership is computed rather than self-declared, and the whole pipeline (validator, distance refutation gate, exact certifier, scorer, static-site generator) lives in the repository and runs on every pull request. We describe the trust model, the pipeline, and the current board, and argue it is a reusable pattern for reproducible, self-certifying benchmarks in quantum error correction.

1 Motivation

Quantum error correction descends from the first codes [1] and the CSS [2, 3] and stabilizer [4] frameworks. Recent qLDPC constructions [5, 6], from the bivariate-bicycle “gross” code $[[144, 12, 12]]$ [7] to planar open-boundary families [8], have made order-of-magnitude reductions in the qubit overhead of fault tolerance plausible. Progress is fast and spread across many groups, raising a shared question: for a given hardware constraint (check weight, locality, size), what is the best known code, and can we trust the claim?

Two facts make this hard to answer in the literature alone. First, results live in heterogeneous tables across papers, with no common format or single place to compare like with like. Second, and more seriously, the headline parameter, distance d , is the one that is hard to verify. Finding a low-weight logical operator gives only an upper bound on d ; proving $d \geq D$ is the NP-hard direction [9]. Reported distances are frequently decoder-based upper bounds, and a heuristic search that fails to find the lightest logical overstates the distance, sometimes by large factors, until a deeper search turns up a lighter operator. A leaderboard that simply trusts submitted numbers would accumulate quiet errors.

The QEC Challenge addresses both problems with quantum-software tooling rather than a static document. Every entry is a small machine-readable file that is automatically re-verified,

distance claims must carry a checkable witness, and a distance refutation step actively searches for counterexamples. The result is a reproducible, community-extensible, self-certifying record. This matters doubly because automated code discovery already works: an LLM-guided evolutionary search recently screened $\sim 2 \times 10^5$ bivariate-bicycle candidates and reported hundreds of codes in one campaign [10], reinforcement learning has cut the qubit overhead of low-weight codes by one to two orders of magnitude [11], and LLM-driven concept evolution reaches constructions, such as non-abelian families, outside the standard ansätze [12]. Such searches propose candidate codes far faster than humans can referee them, and an overstated distance is precisely their characteristic failure mode. An adversarial, machine-checked gate closes that loop, so the board doubles as the verification harness of an automated research cycle: an agent proposes, the pipeline refutes or verifies, and only survivors enter the record.

Related resources catalog codes and bounds but do not automatically verify contributions. Grassl’s tables of best-known code parameters [13] and the Error Correction Zoo [14] are invaluable curated references. The QEC Challenge is complementary: it fixes a machine-checkable submission format and puts an adversarial verifier in the loop, so entries are re-derived and distance claims are actively refuted rather than curated by hand.

2 Design

Three principles shape the system.

(i) Everything cheap is trustless and mandatory. The verifier recomputes n , the logical dimension $k = n - \text{rank}_{\mathbb{F}_2}(H_X) - \text{rank}_{\mathbb{F}_2}(H_Z)$, CSS commutation $H_X H_Z^\top = 0$ over $\mathbb{F}_2$, the maximum check weight, and the geometric locality class, all from the submitted parity checks. A wrong claimed k or a non-commuting “stabilizer” is rejected in milliseconds. Nothing in this tier is taken on trust.

(ii) Distance is layered by how much is proven. Because exact distance does not scale, the board separates three confidence tiers. An *upper bound* ($d \leq$) is backed by an explicit logical-operator witness that the verifier confirms is a genuine nontrivial logical of the claimed weight, so the bound is checkable with no trust required. A *corroborated* bound ($d \leq^*$) is an upper bound that an independent search has tried and failed to beat. An *exact* value ($d =$) is one a server-side integer program has proven minimal. Nothing is ever silently upgraded.

(iii) A code is not a single number. The primary tracks are a grid of two nested, *computed* axes: a locality class (`local-2d-single` $\subset$ `local-2d-bilayer` $\subset$ `unrestricted`, derived from the layout) and a check-weight class (`weight-4` $\subset$ `weight-6` $\subset$ `weight-8` $\subset$ `any`). Membership is derived from the parity checks and the layout, so a track cannot be claimed by relabeling. Within each cell the ranking is a Pareto frontier over (n, k, d, w) ; there is no single winner. The construction family (bivariate bicycle, generalized bicycle, two-block group algebra, tile, topological, ...) cannot be recovered from the matrices, so it is a filterable tag, never a ranking.

3 The verification pipeline

The pipeline is the reusable artifact. On every pull request, GitHub Actions runs, in order:

1. **Structural and trustless checks.** Schema validity, qubit indices in range, no repeated index within a check, CSS commutation, recomputed k matching the claim, $k \geq 1$, a controlled family

and novelty vocabulary, and the witness check: each distance witness must have weight equal to the claimed value, lie in the kernel of the opposite checks, and be a nontrivial logical (not a stabilizer product). This certifies the per-side upper bound.

2. **Computed locality class.** If a layout is present, the verifier derives the locality class from the coordinates, enforcing no cramming (at most `layers` qubits per site, distinct sites at least unit-spaced) and a bounded interaction radius, so a small radius cannot be faked by collapsing qubits.
3. **Distance refutation gate.** Independent searches, a random-information-set search and a syndrome-decoder search, look for a logical lighter than the claimed distance. Any hit is a sound refutation (it exhibits a checkable counterexample). Frontier-advancing claims additionally face an accelerated information-set search at roughly $150\times$ the baseline trial budget that only *proposes* candidates: a find counts as a refutation solely after the trusted verifier validates the witness, so the extra depth adds no trust surface. The per-run seed is random by design so an over-claim cannot be tuned to evade one fixed search, and the gate fails closed. A weekly whole-board sweep re-runs the refutation with fresh seeds to accumulate coverage over time.
4. **Identity and duplicate checks.** The pull-request author must be a listed author of the codes they add. Every code is fingerprinted two ways: the reduced-row-echelon form of (H_X, H_Z) pins the exact stabilizer group (an identical fingerprint is a hard error), and a permutation-invariant Weisfeiler–Leman signature on the Tanner graph flags possible equivalents for review.

A separate, maintainer-run exact tier upgrades a claim to $d =$. For each logical generator t_L it asks, via a bounded integer program (scipy/HiGHS, no commercial solver), whether a v exists with $Hv \equiv 0$, $\langle v, t_L \rangle \equiv 1$, and $\text{wt}(v) \leq \text{value} - 1$. If every generator on both sides is infeasible, the distance is exact; if a lighter logical is found, the claim is refuted; if the solver times out, the side honestly remains an upper bound. A certificate written to `certs/` is what promotes the board label.

4 Metric and the state of the board

The figure of merit the 2D-locality literature uses, kd^2/n , is the Bravyi–Poulin–Terhal saturation ratio [15]. For a 2D-local or bounded-weight code it is bounded by a constant that depends only on the interaction range, check weight, and on-site dimension, not on n , so maximizing it within a fixed locality model is well posed. For asymptotically good codes [16], where k and d both grow with n , it grows like n^2 without bound. The board therefore treats kd^2/n as a per-cell, sortable column and compares like with like, never as a global record. Table 1 lists reference points.

Code	$[[n, k, d]]$	topology	w	kd^2/n
Rotated surface [17]	$[[d^2, 1, d]]$	planar	4	1
Gross code [7]	$[[144, 12, 12]]$	periodic	6	12
Tile code [18]	$[[512, 18, 19]]$	planar	8	12.7
Generalized bicycle [19]	$[[126, 28, 8]]$	periodic	10	14.2
Coset 2BGA [20]	$[[168, 20, 14]]$	unrestricted	8	23.3

Table 1: Reference points spanning the efficiency range; all values are the published parameters.

At the time of writing the board holds 47 verified codes, 25 submitted through the challenge and 22 literature baselines, with 29 certified exact, across the bivariate-bicycle, generalized-bicycle (including non-abelian two-block group-algebra), coset, tile, and topological families. Recent entries

include the board’s first odd- k code, a $[[240, 13, 15]]$ two-block code on the symmetric group S_5 (odd k is impossible for abelian two-block constructions), and a $[[216, 15, 11]]$ planar bilayer code that now leads the 2D-local weight-8 cell. Both were machine-discovered; the submission schema records the producing AI system in a version-specific `model` field, so the provenance of machine-found codes is part of the record. It is seeded against a selection of published codes (the planar open-boundary codes [8], the bivariate-bicycle gross code [7], the generalized-bicycle codes [19, 21, 22], and the surface, toric, and Steane codes [17, 3]), so a submission is always measured against known codes. The seed set is selected, not an exhaustive snapshot, and the board states this: an on-board leader may still be matched by a published code that has not been seeded. The badges and figures are regenerated from the live board on every build, so they track the current numbers rather than a hand-typed count.

5 Relevance to quantum software, and the talk

The challenge is a case study in software infrastructure for quantum error correction: it encodes a trust model directly in CI, with a verifiable file as the submission unit, a locally runnable script as the referee, and a pull request as the contribution workflow. The verifier and refutation gate are reusable beyond this board. The autoresearch loop of Section 1 is not hypothetical: in one recent agent-driven sweep, deep self-refutation overturned five of seven staged frontier candidates before submission, and only the survivors reached the board.

The lightning talk will demonstrate the end-to-end flow (submit, CI verifies, board updates), the layered scheme and its trust model, and how to contribute, by hand or by agent. It will also raise the questions the board surfaces: whether planar weight-6 codes can close the gap to periodic records, how far the refutation gate and exact certification scale, and which families have unmined finite-size headroom.

References

- [1] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4):R2493–R2496, 1995.
- [2] A. Robert Calderbank and Peter W. Shor. Good quantum error-correcting codes exist. *Physical Review A*, 54(2):1098–1105, 1996. arXiv:quant-ph/9512032.
- [3] Andrew M. Steane. Error correcting codes in quantum theory. *Physical Review Letters*, 77(5):793–797, 1996. arXiv:quant-ph/9601029.
- [4] Daniel Gottesman. *Stabilizer Codes and Quantum Error Correction*. PhD thesis, California Institute of Technology, 1997. arXiv:quant-ph/9705052.
- [5] Nikolas P. Breuckmann and Jens Niklas Eberhardt. Quantum low-density parity-check codes. *PRX Quantum*, 2(4):040101, 2021. arXiv:2103.06309.
- [6] Jean-Pierre Tillich and Gilles Zémor. Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength. *IEEE Transactions on Information Theory*, 60(2):1193–1202, 2014. arXiv:0903.0566.
- [7] Sergey Bravyi, Andrew W. Cross, Jay M. Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J. Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627:778–782, 2024. arXiv:2308.07915.
- [8] Zijian Liang, Jens Niklas Eberhardt, and Yu-An Chen. Planar quantum low-density parity-check codes with open boundaries, 2025. arXiv:2504.08887.
- [9] Upendra Kapshikar and Srijita Kundu. On the hardness of the minimum distance problem of quantum codes. *IEEE Transactions on Information Theory*, 69(10), 2023. arXiv:2203.04262.
- [10] Juan Cruz-Benito, Andrew W. Cross, David Kremer, and Ismael Faro. Evolutionary discovery of bivariate bicycle codes with LLM-guided search, 2026. arXiv:2606.02418.
- [11] Austin Yubo He and Zi-Wen Liu. Discovering highly efficient low-weight quantum error-correcting codes with reinforcement learning, 2025. arXiv:2502.14372.

- [12] Zidu Liu and Florian Marquardt. Large-language-model discovery of quantum LDPC codes through structured concept evolution, 2026. arXiv:2606.24808.
- [13] Markus Grassl. Bounds on the minimum distance of linear codes and quantum codes. <http://www.codetables.de>, 2007. accessed 2026.
- [14] Victor V. Albert and Philippe Faist. Handbook of error-correcting codes. <https://errorcorrectionzoo.org>, 2026. arXiv:2606.11484; the Error Correction Zoo.
- [15] Sergey Bravyi, David Poulin, and Barbara Terhal. Tradeoffs for reliable quantum information storage in 2D systems. *Physical Review Letters*, 104(5):050503, 2010. arXiv:0909.5200.
- [16] Pavel Panteleev and Gleb Kalachev. Asymptotically good quantum and locally testable classical LDPC codes. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 375–388, 2022. arXiv:2111.03654.
- [17] A. Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003. arXiv:quant-ph/9707021.
- [18] Vincent Steffan, Shin Ho Choe, Nikolas P. Breuckmann, Francisco Revson Fernandes Pereira, and Jens Niklas Eberhardt. Tile codes: High-efficiency quantum codes on a lattice with boundary, 2025. arXiv:2504.09171.
- [19] Pavel Panteleev and Gleb Kalachev. Degenerate quantum LDPC codes with good finite length performance. *Quantum*, 5:585, 2021. arXiv:1904.02703.
- [20] Arda Aydin, Itzhak Tamo, and Alexander Barg. Breaking the bicycle frame: Coset-based quantum LDPC codes, 2026. arXiv:2606.17268.
- [21] Renyu Wang and Leonid P. Pryadko. Distance bounds for generalized bicycle codes. *Symmetry*, 14(7):1348, 2022. arXiv:2203.17216.
- [22] Hsiang-Ku Lin and Leonid P. Pryadko. Quantum two-block group algebra codes. *Physical Review A*, 109(2):022407, 2024. arXiv:2306.16400.